



THE ARCHITECT'S FIELD GUIDE

50 Real-World System Design Trade- Offs

Interview Preparation Framework · 50-Item Cheat Sheet ·
13 Full Architectural Blueprints

"Every architecture decision is a trade-off — the skill is knowing which one to make, and why. This guide turns that skill into a repeatable interview framework."

URL Shortener

Rate Limiter

Payment Gateway

Distributed Cache

Uber Location

Video Streaming

Chat App

News Feed

Notifications

Web Crawler

Autocomplete

E-commerce

Job Scheduler

C R E A T E D B Y

Kartik Mungole

@kartikmungole

Table of Contents

From interview framework to the 50 trade-offs cheat sheet

This is a **69-page** guide. Part A builds your interview method. Part B is the 50-item trade-off reference. Part C applies it across **13 full production-style blueprints** (see TOC pages 2-5), followed by a mock interview transcript and a 30-question practice bank.

PART A — INTERVIEW PREPARATION

01	How System Design Interviews	06
	Actually Work	
	Format, levels, what's really being evaluated	
02	The RESHADED Framework & 45-Minute Timeline	08
	A repeatable structure for any design question	
03	Back-of-Envelope Math — Numbers to Know	10
	Latency table, throughput, storage sizing cheat sheet	
04	Common Mistakes & How You're Actually Graded	12
	Signal vs. noise from the interviewer's side of the table	

PART B — FOUNDATIONS & THE 50 TRADE-OFFS

05	Foundations I — CAP Theorem & PACELC	14
	The two laws every trade-off traces back to	

Table of Contents

Foundations and the start of the 50 trade-offs cheat sheet

PART B – FOUNDATIONS & THE 50 TRADE-OFFS (cont'd)

06 Foundations II — Consistency	16
Models & Scaling Patterns	
Strong/eventual/causal · vertical vs horizontal · sharding	
07 The 50 Trade-Offs — Data & Caching	18
9 storage trade-offs + 8 caching trade-offs, 1 page each	
08 The 50 Trade-Offs — Scale & Communication	20
8 scalability trade-offs + 8 messaging trade-offs	
09 The 50 Trade-Offs — Reliability & Security	22
9 fault-tolerance trade-offs + 8 security/ops trade-offs	

PART C – 13 FULL ARCHITECTURAL BLUEPRINTS

10 URL Shortener	24
Key generation, hot-key caching, redirect strategy	
11 Rate Limiter	27
Token bucket vs sliding window, distributed enforcement	

Table of Contents

The 13 blueprints — each spans 3 pages: Requirements/HLD, then two Detail pages

PART C – 13 FULL ARCHITECTURAL BLUEPRINTS (cont'd)

12 Payment Gateway	30
Idempotency, ledger consistency, Saga orchestration	
13 Distributed Caching System	33
Consistent hashing, eviction, cache stampede	
14 Uber Driver Location Tracking	36
Geospatial indexing, high-frequency writes	
15 Video Streaming Platform	39
Transcoding pipeline, adaptive bitrate, CDN tiers	
16 Chat / Messaging App (WhatsApp-style)	42
Delivery guarantees, ordering, group fan-out	
17 News Feed (Twitter / Instagram-style)	45
Fan-out on write vs read, the celebrity problem	
18 Notification System	48
Multi-channel delivery, retries, user preferences	
19 Web Crawler	51
URL frontier, politeness, dedup at scale	

Table of Contents

The rest of the blueprints, mock interview, and practice bank

PART C – 13 FULL ARCHITECTURAL BLUEPRINTS (cont'd)

20 Search Autocomplete / Typeahead	54
Trie design, precompute vs real-time trending	
21 E-Commerce Order & Inventory System	57
Mixed CAP zones in one product, oversell prevention	
22 Distributed Job Scheduler / Message Queue	60
Delayed jobs, leader election, exactly-once debate	

PART D – PRACTICE

23 Mock Interview Transcript — "Design a News Feed"	63
The RESHADED framework applied live, start to finish	
24 30-Question Practice Bank	66
Categorized by difficulty, with the core trade-off to spot	
25 Closing Notes & 4-Week Study Plan	69
How to actually turn this into interview-day fluency	



Each blueprint follows the same three-page rhythm: page 1 covers Requirements, Estimation, High-Level Design and the API; pages 2-3 go deep on the trade-offs that actually get asked as interview follow-ups. Read blueprints in order the first time through — the guide deliberately reuses patterns (Saga, idempotency, consistent hashing) across systems so they compound.

How System Design Interviews Actually Work

Understand the format before you try to win it

Q What is the interviewer actually testing — do I need to "know the answer"?

A There usually isn't one "correct" architecture, so you're not being graded against a hidden answer key. You're graded on **process**: do you clarify scope before designing, do you make deliberate trade-offs instead of guessing, can you go deep on a component when pushed, and can you justify decisions against stated requirements. A mediocre design defended with clear reasoning consistently beats a "textbook" design recited without justification.

Q How does the bar change by seniority level?

A

Level	What's expected
Mid-level	Can design a single well-scoped service correctly, knows common patterns, needs some prompting to go deep
Senior	Drives the conversation, proactively surfaces trade-offs, estimates capacity unprompted, discusses failure modes
Staff+	Reasons about organizational/team boundaries, cost, migration paths, and cross-system trade-offs

How System Design Interviews Actually Work — continued

Formats you'll see, and the signal that separates strong answers

Q What are the most common interview formats?

A

OPEN-ENDED DESIGN

- "Design Twitter" / "Design WhatsApp"
- You drive scope, estimation, and depth
- Most common at mid-to-senior level

OBJECT-ORIENTED / LOW-LEVEL DESIGN

- "Design a parking lot" in code/classes
- Tests OOP principles, not distributed systems
- Common in earlier rounds

 This guide focuses on the open-ended, distributed-systems style — used at almost every mid-to-large tech company for backend/infra roles.

Q What's the single biggest signal that separates a strong answer from a weak one?

- A **Whether the design changes when the requirements change.** A candidate who proposes the same architecture regardless of scale, consistency needs, or budget hasn't understood that system design *is* trade-off reasoning. A strong candidate visibly updates the design — "if writes are 10x reads instead, I'd switch from read replicas to write-optimized sharding" — because that's proof the trade-offs were understood, not memorized.

The RESHADED Framework

One structure you can run on literally any design question

R Requirements

Clarify functional (what must it do) and non-functional (scale, latency, consistency, availability) requirements. Never assume — ask.

E Estimation

Back-of-envelope: users, QPS (read/write), storage growth, bandwidth. This number set drives every later decision.

S Storage Schema

Define the core entities and how they're stored — this is where SQL vs. NoSQL (trade-off #1) gets decided.

H High-Level Design

Draw the box diagram: client → gateway → services → storage. Keep it simple first, add complexity only when justified.

A API Design

Define the 3-5 core endpoints/contracts. Forces you to nail down exactly what each component owns.

D Detailed Design

Deep-dive into 1-2 components the interviewer cares about most (they'll usually steer you here).

E Evaluate Trade-Offs

Explicitly say what you optimized for and what you gave up — this is where the 50-item cheat sheet earns its keep.

D Diagnose Bottlenecks

Identify the single points of failure and hot spots, and propose how you'd fix them at 10x scale.

The 45-Minute Timeline

Where the framework's eight steps should actually land on the clock

SUGGESTED TIME BUDGET – 45 MINUTE INTERVIEW

Requirements	3m	0-3m
Estimation	5m	3-8m
Storage Schema	4m	8-12m
High-Level Design	9m	12-21m
API Design	4m	21-25m
Detailed Design	12m	25-37m
Trade-offs & Bottlenecks	5m	37-42m
Wrap-up / Q&A	3m	42-45m

⚠ The most common failure mode is spending 25 of your 45 minutes on high-level design and leaving no time for a deep dive — which is usually where most of the scoring signal lives. Timebox yourself out loud: "let's spend 5 more minutes here, then go deeper."

💡 If the interviewer interrupts to steer you into a deep dive earlier than planned, follow them — the timeline is a default, not a script. Reading the room correctly is itself part of the signal.

Back-of-Envelope Math — Latency & Throughput

Rough numbers, memorized once, that make every estimation section fast and credible

LATENCY NUMBERS EVERY ENGINEER SHOULD KNOW

Operation	Approx. latency
L1/L2 cache reference	~1 ns
Main memory (RAM) reference	~100 ns
Redis / in-memory cache GET (same datacenter)	~0.5-1 ms
SSD random read	~100-150 μ s
Round trip within same datacenter	~0.5 ms
DB read (indexed, warm)	~1-5 ms
HDD disk seek	~10 ms
Round trip cross-country / cross-continent	~50-150 ms

THROUGHPUT RULES OF THUMB

Component	Rough capacity (single node)
Redis / Memcached node	~100K-500K simple ops/sec
Relational DB (transactional writes)	~1K-10K writes/sec
Kafka broker	~10s-100s of MB/sec sustained
Web/app server (stateless, simple request)	~1K-10K req/sec
Load balancer (L7)	~10K-100K req/sec

Back-of-Envelope Math — Storage & a Worked Example

Putting the numbers together into a real estimation, live

STORAGE & POWERS OF TWO CHEAT SHEET

Unit	Bytes	Practical intuition
1 KB	10^3	A short paragraph of text
1 MB	10^6	A compressed photo, or ~1M chars
1 GB	10^9	~1000 photos, or a few minutes of HD video
1 TB	10^{12}	~250K photos, or one modern DB node's comfortable working set
1 PB	10^{15}	Requires a distributed storage layer (S3-class object storage)

WORKED EXAMPLE

100M DAU, each posts 2x/day, avg post 300 bytes → 60M writes/day ≈ **~700 writes/sec average**, likely 5-10x that at peak → **~3.5K-7K writes/sec peak**. That single calculation tells you whether a single SQL primary can handle it (yes, comfortably) or whether you need sharding (#24) from day one.

STORAGE GROWTH, SAME EXAMPLE

60M posts/day × 300 bytes ≈ 18 GB/day of raw post data → **~6.5 TB/year**. Comfortably fits a single well-provisioned database with room to grow — sharding driven by write throughput, not raw storage size, in this example.

Common Mistakes & How You're Actually Graded

The gap between a passing and failing answer is rarely technical knowledge

Q What are the top mistakes that sink otherwise-strong candidates?

A

Mistake	Why it hurts	Fix
Jumping straight to a diagram	Signals you don't gather requirements before building	Force 2-3 min of clarifying questions first, every time
Skipping estimation	Later decisions (sharding, caching) are unjustified	Always compute QPS and storage growth, even roughly
Going deep too early	Runs out the clock before the full picture is drawn	Finish the high-level design first; deep-dive after
Silent thinking	Interviewer can't score reasoning they can't hear	Narrate: "I'm choosing X over Y because..."
Over-engineering by default	Reads as poor judgment, not rigor	Match design to the estimated numbers, not FAANG-scale defaults
Never mentioning failure modes	Suggests inexperience with real systems	Proactively raise: "what happens if this node goes down?"

Common Mistakes & Grading — continued

What the scoring rubric actually weighs, and what to do when you're stuck

Q How do interviewers typically score the conversation afterward?

A Most rubrics collapse into four buckets — problem-solving/structure, technical depth, trade-off reasoning, and communication. Notice that **technical depth is only one of four**. A candidate who picks a mediocre database but explains the reasoning clearly and adapts when challenged usually scores higher than one who picks the "correct" database silently.

Q What should I do when I genuinely don't know something the interviewer asks?

A Reason from first principles out loud instead of guessing silently or freezing. "I haven't worked with X directly, but based on how similar systems handle this, I'd expect the trade-off to be..." This still demonstrates the transferable skill being tested — trade-off reasoning — even without direct familiarity with a specific technology.

💡 Interviewers regularly rate "I don't know, but here's how I'd figure it out" above a confidently wrong answer.

Foundations I — CAP Theorem & PACELC

The two laws almost every other trade-off in this guide traces back to

Q What is the CAP theorem, in plain terms?

- A In a distributed system, when a **network partition** happens (and it will), you can only keep two of three guarantees: **Consistency** (every read sees the latest write), **Availability** (every request gets a response), and **Partition tolerance** (the system keeps working despite dropped/delayed messages). Since partitions are unavoidable, the real-world choice is CP vs. AP.

Choice	Means	Example systems
CP	Reject requests rather than return stale data	Banking ledgers, ZooKeeper, etcd
AP	Always respond, even with slightly stale data	DNS, shopping carts, social feeds, DynamoDB (default)

Q What is PACELC and why does it matter more day-to-day?

- A CAP only describes behavior **during a partition** — which is rare. PACELC extends it: **"If Partitioned, choose A or C; Else (normal operation), choose Latency or Consistency."** This is the trade-off you hit every single day, since most requests happen when the network is healthy.

i This is why "eventually consistent" isn't a bug — it's a deliberate latency optimization made almost all the time, not just during outages.

Foundations I — continued

Framing trade-offs out loud, and why one system rarely picks CP or AP globally

Q How should I frame a trade-off out loud in an interview?

A Four steps: **(1) state the constraint** — "we need sub-100ms reads at 50K QPS." **(2) name the two options** — cache-aside vs. read-through. **(3) state what each costs** — one risks stale reads, the other adds a proxy hop. **(4) pick, and justify against the constraint** — not against the technology's popularity.

Q Does one system have to pick CP or AP globally?

A No — and saying so is a strong signal. Most real products are **zoned**: the payment/inventory core is CP, while the product catalog or activity feed is AP. See Blueprint 12 (Payment Gateway) and Blueprint 21 (E-Commerce) later in this guide for exactly this split inside one product.

Foundations II — Consistency Models & Scaling

The vocabulary you need before the 50-item cheat sheet will make full sense

CONSISTENCY MODELS, WEAKEST TO STRONGEST

Model	Guarantee	Typical use
Eventual	All replicas converge <i>eventually</i> , no timing guarantee	Like counts, view counts, DNS
Read-your-writes	A user always sees their own writes immediately	Profile edits, posting a comment
Causal	Causally related operations are seen in order by everyone	Chat threads, comment replies
Strong (linearizable)	Every read sees the latest write, globally, immediately	Bank balances, inventory counts, leader election

SCALING PATTERNS – VERTICAL VS. HORIZONTAL

VERTICAL (SCALE UP)

- Bigger CPU/RAM/disk on one machine
- Zero code changes, simplest to operate
- Hard ceiling, single point of failure

HORIZONTAL (SCALE OUT)

- More machines sharing the load
- Near-unlimited ceiling, better fault tolerance
- Requires statelessness, partitioning

Foundations II — continued

Stateless design and sharding strategies

STATELESS VS. STATEFUL SERVICES (TRADE-OFF #19)

A **stateless** service keeps no per-user session data in memory — any instance can serve any request, so horizontal scaling and failover are trivial. A **stateful** service (e.g., a WebSocket connection holder) ties a client to a specific instance, trading operational simplicity for lower latency and richer real-time behavior. The universal pattern: push state to a shared, external store (Redis, DB) so your compute layer stays stateless wherever possible.

SHARDING STRATEGIES (TRADE-OFF #24)

Strategy	How	Best for	Weakness
Range-based	Partition by key range (A-M, N-Z)	Efficient range scans, time-series	Hot shards if data is skewed
Hash-based	Partition by $\text{hash}(\text{key}) \% N$	Even distribution	Range queries need scatter-gather
Directory-based	Lookup service maps key → shard	Flexible rebalancing	Lookup service is a new dependency

The 50 Trade-Offs Cheat Sheet — Data & Storage

9 trade-offs that decide how and where your data lives

Data & Storage				9 trade-offs
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
1	Data model	SQL (relational)	NoSQL (doc/KV/wide-col)	Data is relational & needs strong ACID guarantees
2	Schema shape	Normalization	Denormalization	Writes > reads, storage cost matters more than read speed
3	Consistency	Strong consistency	Eventual consistency	Money, inventory, or anything that can't be "slightly wrong"
4	Growth strategy	Vertical scale (bigger box)	Sharding (more boxes)	Simplicity matters and data fits one powerful node for now
5	Read scaling	Read replicas	Single writer only	Read traffic vastly exceeds write traffic
6	Storage layout	Row-based storage	Column-based storage	Workload is transactional (OLTP), full-row reads
7	Write path	Write-through (DB first)	Write-back (cache first, async flush)	Data integrity > write latency
8	Replication mode	Synchronous replication	Asynchronous replication	Zero data loss is non-negotiable (finance, health)
9	Search	Elasticsearch / dedicated search index	DB LIKE / native full-text	Search is a core feature at scale, needs ranking & facets

💡 Notice the pattern across every table in this cheat sheet: nearly every "A" trades **latency or simplicity** for **consistency or durability**, and vice versa for "B". That tension is the entire discipline.

The 50 Trade-Offs Cheat Sheet — Caching & Performance

8 trade-offs that decide how fast your system feels

⚡ Caching & Performance				8 trade-offs
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
10	Read path	Cache-aside	Read-through cache	App team wants full control over cache-miss logic
11	Write path	Write-through cache	Write-behind (write-back) cache	Cache and DB must never disagree
12	Eviction	TTL-based expiry	LRU (least recently used)	Data has a natural "freshness window"
13	Cache location	Local (in-process) cache	Distributed cache (Redis/Memcached)	Single-node latency matters more than cross-node consistency
14	Edge delivery	CDN caching	Always hit origin	Content is static/semi-static and globally distributed
15	Startup behavior	Cache warming (pre-load)	Cold start (lazy load)	Predictable traffic spikes (e.g., product launch)
16	Cache medium	In-memory (Redis)	Disk-based cache	Sub-millisecond latency is required
17	Key distribution	Consistent hashing	Modulo hashing	Cache nodes will be added/removed dynamically

The 50 Trade-Offs Cheat Sheet — Scalability & Load

8 trade-offs that decide how your system handles growth

 Scalability & Load 8 trade-offs				
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
18	Scaling axis	Vertical scaling	Horizontal scaling	Early stage, need speed of delivery over elasticity
19	Service design	Stateless services	Stateful services	You need trivial horizontal scaling & failover
20	Load balancing	Round robin	Least connections	Requests are roughly uniform in cost
21	Rate limiting algo	Token bucket	Sliding window log	Bursty traffic should be allowed within a budget
22	Autoscaling	Reactive (metric-based)	Predictive (schedule/ML-based)	Traffic patterns are irregular/unknown
23	Service topology	Monolith	Microservices	Small team, unclear domain boundaries, speed matters most
24	Sharding key	Range-based sharding	Hash-based sharding	You need efficient range queries (e.g., time-series)
25	Pagination	Offset-based	Cursor-based (keyset)	Dataset is small/static, jump-to-page UX is needed

The 50 Trade-Offs Cheat Sheet — Communication & Messaging

8 trade-offs that decide how services talk to each other

 Communication & Messaging 8 trade-offs				
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
26	Call style	Synchronous REST	Asynchronous messaging	Caller needs an immediate response to proceed
27	Update delivery	Push (webhooks)	Pull (polling)	Low latency updates matter and consumer is reachable
28	Fan-out model	Message queue (1-to-1)	Pub/Sub (1-to-many)	Exactly one consumer should process each message
29	Delivery guarantee	At-least-once	Exactly-once	Duplicate processing is safe if consumer is idempotent
30	Protocol	gRPC	REST/JSON	Internal service-to-service, need speed & strict contracts
31	Real-time channel	Long polling	WebSockets	Updates are infrequent, want simpler infra
32	Workflow control	Orchestration (central)	Choreography (event-driven)	Workflow has complex branching that needs visibility
33	Data processing	Batch processing	Stream processing	Latency doesn't matter, throughput/cost efficiency does

The 50 Trade-Offs Cheat Sheet — Reliability & Fault Tolerance

9 trade-offs that decide what happens when something breaks

 Reliability & Fault Tolerance				9 trade-offs
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
34	Error handling	Retry with backoff	Fail fast	Failures are likely transient (network blips)
35	Downstream protection	Circuit breaker	No protection	A failing dependency could cascade into an outage
36	Duplicate safety	Idempotency keys	Accept duplicate risk	Retries could double-charge or double-process
37	Failover model	Active-active	Active-passive	You need zero-downtime failover and can afford dual capacity
38	Distributed transactions	Two-phase commit (2PC)	Saga pattern	Strict atomicity across a small, tightly-coupled set of services
39	Leader selection	Consensus (Raft/Paxos)	Manual/static failover	Automatic, correct leader election under failure is required
40	Isolation	Bulkhead pattern	Shared resource pool	One noisy tenant/feature must not starve the rest
41	Degraded state	Graceful degradation	Hard failure	Partial functionality is better than none (e.g., stale recs)
42	Overload handling	Backpressure	Unbounded queue	Producers can slow down without breaking correctness

The 50 Trade-Offs Cheat Sheet — Security, Auth & Ops

8 trade-offs that decide how safely and smoothly you ship

🔒 Security, Auth & Ops				8 trade-offs
#	TRADE-OFF	OPTION A	OPTION B	CHOOSE A WHEN...
43	Auth token	JWT (stateless)	Session tokens (stateful)	You need to scale auth checks horizontally without shared state
44	API access	OAuth2	Static API keys	Third-party/delegated access with scoped permissions is needed
45	Data protection	Encryption at rest	Skip for performance	Data is sensitive (PII, financial, health)
46	Rate limit scope	Per-user limiting	Per-IP limiting	You can reliably identify authenticated users
47	Entry point	API Gateway	Direct service exposure	You need centralized auth, rate limiting, and routing
48	Deployment	Blue-green deployment	Rolling deployment	Instant rollback matters more than resource cost
49	Release control	Feature flags	Direct release	You need to de-risk releases / gradual rollout or A/B tests
50	Observability	Rich logs+traces (higher cost)	Metrics only (cheaper)	Deep debuggability of complex distributed flows is required

💡 These six pages cover the majority of "how would you design X" follow-up questions. Each blueprint ahead references these trade-offs by number.

URL Shortener

A deceptively simple system that's 95% read traffic and 100% about key generation

REQUIREMENTS

FUNCTIONAL

- Given a long URL, return a short unique alias
- Given a short alias, redirect to the original URL
- Optional: custom aliases, expiration, analytics

NON-FUNCTIONAL

- Redirect latency <100ms
- Highly available (redirects can't go down)
- Codes must be unique, ideally unguessable

ESTIMATION

100:1

READ : WRITE RATIO

7 chars

BASE62 CODE LENGTH

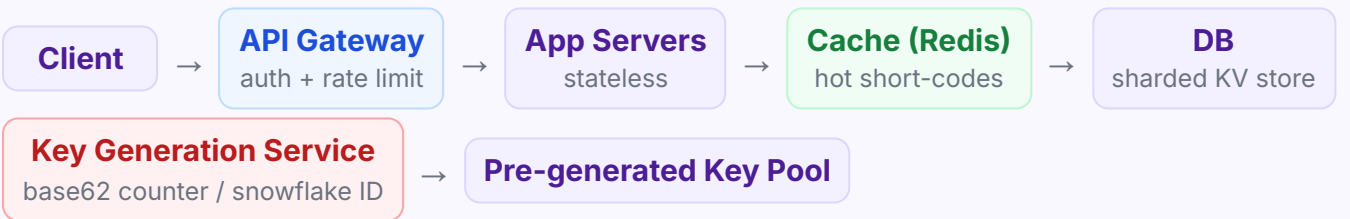
~3.5T

UNIQUE URLS (62^7)

<100ms

REDIRECT LATENCY
TARGET

HIGH-LEVEL DESIGN



API DESIGN

Endpoint	Purpose
POST /api/v1/shorten {long_url, custom_alias?, ttl?}	Creates a short code, returns short URL
GET /{short_code}	302 redirect to the original long URL
GET /api/v1/analytics/{short_code}	Returns click-count and referrer stats

URL Shortener — Detailed Design & Trade-Offs

Where the interview signal actually lives

Q How do you generate the short code — hash the URL or use a counter?

A Hashing (MD5/SHA + truncate) is simple but causes collisions that need retry logic. A **distributed counter** (or Snowflake-style ID) encoded in base62 guarantees uniqueness with zero collisions, at the cost of needing a coordinated ID-generation service. Production systems typically pre-generate a pool of unique keys in batches and hand them out from a key-generation service, avoiding a single point of contention.

Q Why is caching (#13, #16) the single most important decision here?

A Traffic follows a power law — a small fraction of short URLs (viral links) account for most redirects. A **distributed in-memory cache (Redis)** in front of the DB for the hot keys turns a DB-bound system into a cache-bound one, cutting p99 latency from ~40ms to ~2ms.

⚠ Watch for the "hot key" problem: one viral link can overload a single cache shard. Mitigate with client-side caching / CDN edge redirects for extremely hot links.

Q SQL or NoSQL (#1) for the URL mapping table?

A The access pattern is a pure key-value lookup (short_code → long_url) with no joins — a textbook case for **NoSQL (DynamoDB/Cassandra)**, which scales horizontally by sharding on the short code with predictable latency.

URL Shortener — Detailed Design & Trade-Offs, continued

Redirect codes and where the bottleneck moves at 10x scale

Q 301 or 302 redirect — does it matter?

A **301 (permanent)** lets browsers cache the redirect, cutting server load, but you lose the ability to track every click. **302 (temporary)** forces every click through your server (full analytics) at the cost of higher load. Most link-shortener products use 302 specifically to preserve click analytics.

Q Where's the bottleneck at 10x scale, and how do you fix it?

A The cache layer becomes the bottleneck first. Fix with **consistent hashing (#17)** across more cache nodes, and a two-tier cache (local in-process LRU + distributed Redis) so the hottest handful of keys never leave a single app server's memory.

Rate Limiter

Protecting your system from itself — and from everyone else

REQUIREMENTS

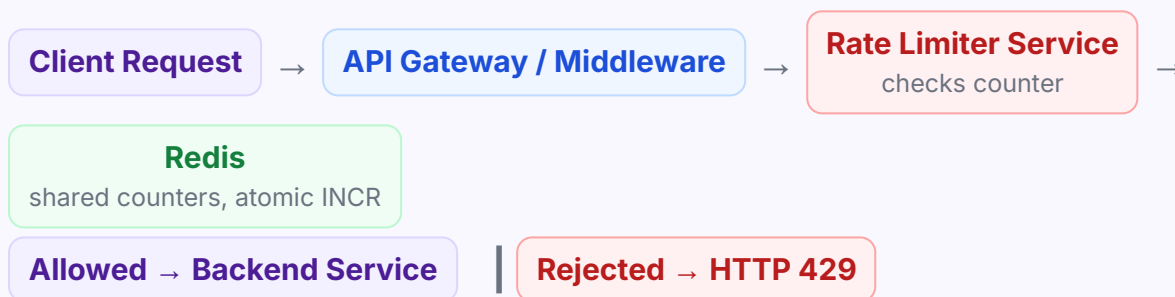
FUNCTIONAL

- Limit requests per user/IP/API key to N per window
- Return clear rejection (HTTP 429) with retry-after
- Configurable limits per endpoint/tier

NON-FUNCTIONAL

- Must add minimal latency (<5ms) to every request
- Must be accurate across many gateway instances
- Must degrade safely if the limiter itself fails

HIGH-LEVEL DESIGN



ALGORITHM COMPARISON (TRADE-OFF #21)

Algorithm	How it works	Pros	Cons
Token Bucket	Bucket refills at fixed rate; each request consumes a token	Allows controlled bursts; memory-efficient	Slightly more complex to reason about
Leaky Bucket	Requests queue and are processed at a fixed output rate	Smooths traffic to a constant rate	Bursts get queued/delayed, not rejected instantly
Fixed Window	Count requests in a fixed time window	Very simple to implement	Allows 2x burst at window boundaries
Sliding Window Log	Store timestamp of every request, count within rolling window	Most accurate — no boundary bursts	Higher memory (stores every timestamp)

Rate Limiter — Detailed Design & Trade-Offs

Enforcing one true limit across a fleet of stateless gateways

Q Per-IP or per-user rate limiting (#46)?

- A Use **both, layered**: per-IP as a coarse first line of defense against unauthenticated abuse, and per-user/per-API-key limits post-authentication for fair usage — since many real users can share one IP (NAT, corporate networks) and one attacker can rotate IPs.

Q How do you enforce limits consistently across 50 gateway instances?

- A This is trade-off **#13 (local vs. distributed cache)** in action. A local in-memory counter per instance lets a user get 50x their real limit (one bucket per instance). A **shared Redis counter** with atomic `INCR` + `EXPIRE` gives one true global count at the cost of a network hop per request.

```
INCR user:123:count
EXPIRE user:123:count 60 (if first increment)
IF count > limit: return 429
```

Rate Limiter — Detailed Design & Trade-Offs, continued

Failure behavior and dynamic, tiered limits

Q What happens if the rate limiter itself goes down?

- A Trade-off **#41 (graceful degradation vs. hard failure)**. Failing open (let all traffic through) risks an overload cascade; failing closed blocks all traffic over a non-critical component. Most systems **fail open with a local fallback rate limit** — better to briefly under-protect than take the whole API down.

Q How would you support per-endpoint, tiered limits (free vs. paid users)?

- A Store rate-limit config (limit, window, algorithm) in a small, cacheable config store keyed by `{tier, endpoint}`, refreshed on a TTL rather than looked up per-request — a direct application of **cache-aside (#10)** to keep the hot path fast while still allowing dynamic limit changes without a redeploy.

Payment Gateway

Where correctness always wins — money can never be "eventually consistent"

REQUIREMENTS

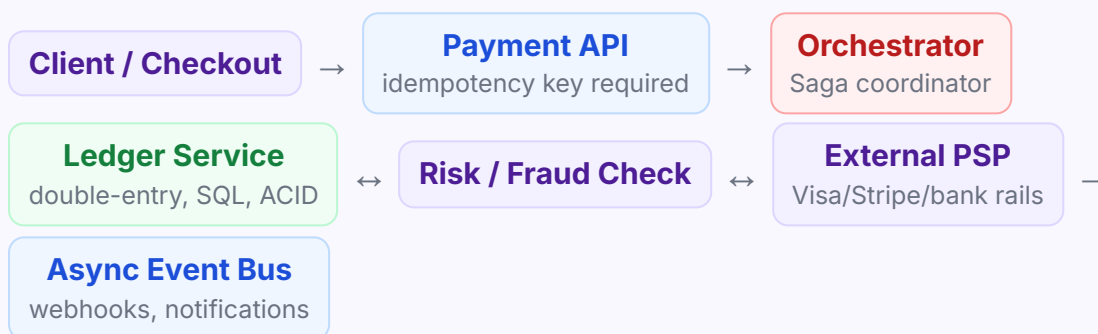
FUNCTIONAL

- Accept a charge request, move money via external PSP
- Guarantee no double-charge on retry
- Support refunds and reconciliation

NON-FUNCTIONAL

- Strong consistency on the ledger, always
- PCI-DSS compliant handling of card data
- Auditable, replayable event history

HIGH-LEVEL DESIGN



API DESIGN

Endpoint	Purpose
POST /charges Idempotency-Key: <uuid>	Initiates a charge; safe to retry with the same key
POST /refunds {charge_id, amount}	Issues a full or partial refund
GET /charges/{id}	Returns current status of a charge (pending/succeeded/failed)

Payment Gateway — Detailed Design & Trade-Offs

Correctness-first design in a world of unreliable networks

Q Why does a payment gateway have to pick CP, not AP?

- A Because a lost or double debit is unacceptable. The core ledger uses a **strongly consistent SQL database** with ACID transactions (#1, #3), even though this costs latency. Non-critical paths — a receipt email, a loyalty-points dashboard — can be eventually consistent and async, so strict consistency is scoped tightly to the money-moving core.

Q Why is idempotency (#36) the single most critical trade-off here?

- A Networks fail, clients retry, mobile apps double-tap "Pay Now." Every payment request carries a client-generated **idempotency key**; the server stores the result of the first attempt and returns the identical response for any duplicate — turning an unsafe retry into a safe no-op.

IDEMPOTENCY FLOW

POST /charge Idempotency-Key: abc123 1st call → processes charge, stores result under abc123 2nd call → same key seen → returns cached result, no new charge

Payment Gateway — Detailed Design & Trade-Offs, continued

Multi-step flows and compliance-driven encryption

Q 2PC or Saga pattern (#38) for a multi-step payment flow?

A Two-phase commit would require locking wallet, fraud-check, and external-PSP services simultaneously — impossible across a PSP you don't control. The standard answer is a **Saga**: each step commits locally and publishes an event; if a later step fails, **compensating transactions** (refund, release hold) undo earlier steps.

Q Encryption at rest (#45) — worth the overhead here?

A Non-negotiable. PCI-DSS mandates encrypting card data at rest and in transit; gateways typically avoid storing raw card numbers at all, using **tokenization** so the cost of encryption is paid once, at the vault boundary, not on every service that touches an order.

Distributed Caching System

Building the cache layer itself — like designing Redis or Memcached from scratch

REQUIREMENTS

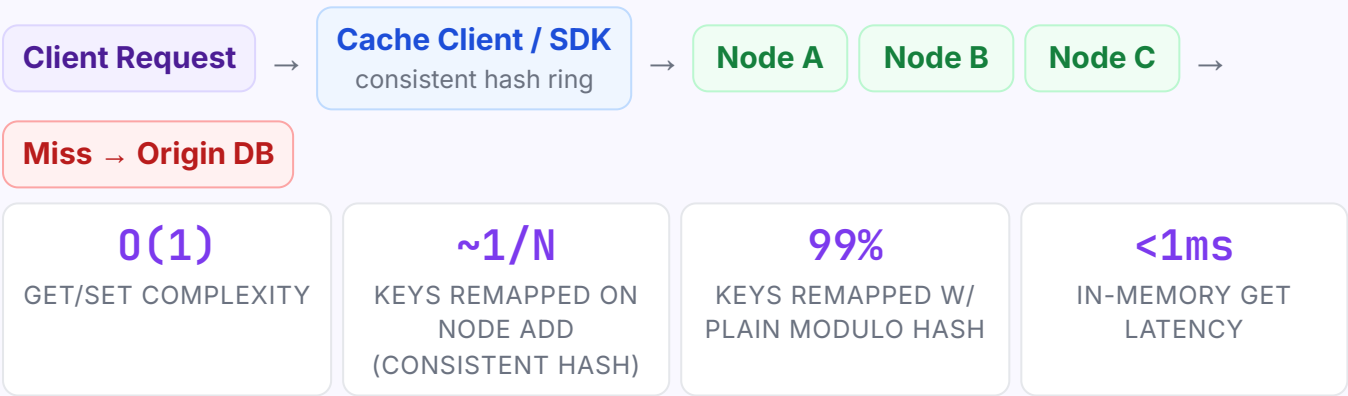
FUNCTIONAL

- GET/SET/DELETE with sub-millisecond latency
- Automatic eviction under memory pressure
- Nodes can be added/removed without a full re-shuffle

NON-FUNCTIONAL

- High availability, tolerate node failure
- Predictable, low-variance latency (p99 matters most)
- Even key distribution across nodes

HIGH-LEVEL DESIGN



Distributed Caching System — Detailed Design & Trade-Offs

The two decisions that make or break a cache layer at scale

Q Consistent hashing vs. modulo hashing (#17) — why does it matter so much?

A With plain modulo (`hash(key) % N`), adding or removing one node remaps **almost every key** — a cache-wide stampede to the origin DB. **Consistent hashing** maps nodes and keys onto a ring; adding/removing a node only reshuffles roughly `1/N` of all keys. This is the single most important decision in any distributed cache.

Q LRU vs. TTL eviction (#12) — how do you decide?

A **LRU** answers "what's least likely to be needed again soon" — ideal when memory is the constraint. **TTL** answers "when does this data become stale" — essential when correctness matters. Production systems typically run **both together**: TTL guarantees staleness bounds, LRU handles memory pressure before TTL expires.

Distributed Caching System — Detailed Design & Trade-Offs, continued

Stampede protection and replication for availability

Q What is "cache stampede" and how do you design around it?

- A When a hot key expires, thousands of concurrent requests can all miss the cache at once and hammer the origin DB simultaneously — an extension of trade-off [#42 \(backpressure\)](#). Mitigations: a [mutex/lock](#) so only one request repopulates the cache; [probabilistic early expiration](#); or serving slightly stale data while one background refresh happens.

💡 "Write-through everything, TTL of zero" sounds safe but removes the cache's shock-absorber role entirely — a common trap.

Q How do you handle replication for high availability (#37)?

- A Each shard typically has a [primary + 1-2 replicas](#) using async replication ([#8](#)) — losing a few milliseconds of the most recent writes on failover is acceptable for a cache, since the source of truth remains the origin DB.

Uber Driver Location Tracking

Millions of GPS pings a second, a geospatial index, and a latency budget of milliseconds

REQUIREMENTS

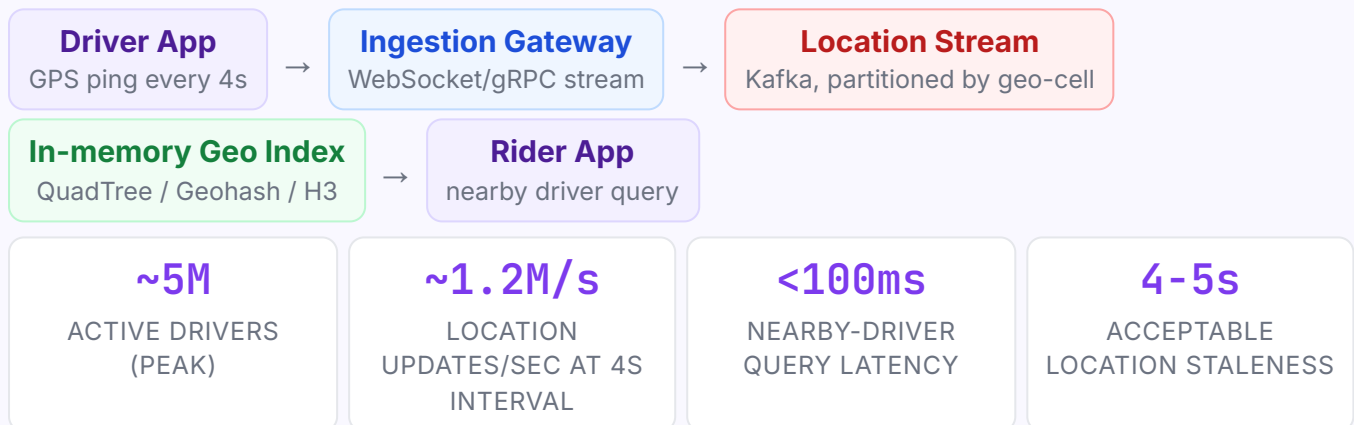
FUNCTIONAL

- Drivers periodically report location
- Riders can query "drivers within N km"
- Match riders to the nearest available driver

NON-FUNCTIONAL

- Extremely write-heavy (constant GPS pings)
- Query latency <100ms
- A few seconds of staleness is acceptable

HIGH-LEVEL DESIGN



Uber Driver Location Tracking — Detailed Design & Trade-Offs

A write-first system where eventual consistency is a deliberate feature

Q Why is this fundamentally a write-heavy, not read-heavy, system?

A Every active driver pushes a GPS update every few seconds regardless of demand, while "find nearby drivers" queries only fire when a rider opens the app. This flips the usual read-heavy assumption and drives the whole design toward an **in-memory, write-optimized geospatial index** — a plain SQL table with lat/long columns would collapse under this write volume.

Q How do you index location for fast "drivers within 2km" queries?

A	Approach	How	Trade-off
	Geohash	Encode lat/long into a string prefix	Simple, but boundary edge-cases near prefix borders
	QuadTree	Recursively divide map into 4 quadrants	Naturally adapts density (dense cities = smaller cells)
	H3 (hexagonal)	Uber's own hex-grid spatial index	Uniform neighbor distance, no geohash edge artifacts

Uber Driver Location Tracking — continued

Consistency choice and the ingestion pipeline

Q Strong or eventual consistency (#3) for driver location?

A **Eventual, deliberately.** A driver's position 3 seconds ago is still useful for matching; waiting for a globally consistent write would add latency for no benefit. A clean PACELC example: under normal operation, the system explicitly chooses **latency over consistency** because the cost of staleness is negligible.

Q Stream (Kafka) or direct DB writes (#33) for ingesting pings?

A A message stream, partitioned by geographic cell, decouples the high-volume ingestion rate from downstream consumers — the geo-index updater, analytics, ETA recalculation — so a slow consumer never blocks the write path.

Video Streaming Platform

From a single uploaded file to millions of concurrent adaptive-bitrate streams

REQUIREMENTS

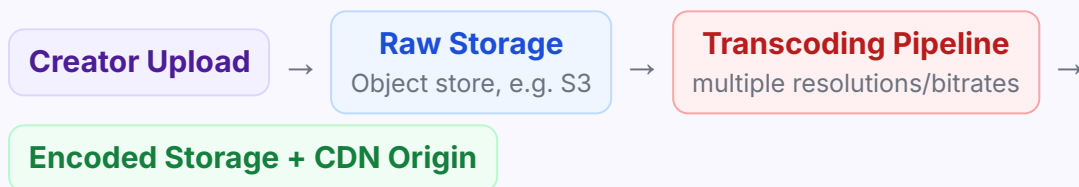
FUNCTIONAL

- Creators upload video; it gets processed and published
- Viewers stream video adaptively to their bandwidth
- Track views, likes, comments

NON-FUNCTIONAL

- Playback must start fast (<2s) and rarely buffer
- Massive read (view) volume vs. modest write (upload) volume
- Global audience — latency depends on geography

HIGH-LEVEL DESIGN — UPLOAD PATH



HIGH-LEVEL DESIGN — PLAYBACK PATH



Video Streaming Platform — Detailed Design & Trade-Offs

The internet's most bandwidth-hungry content type, delivered reliably

Q Why transcode into multiple resolutions instead of streaming the original file?

A Adaptive bitrate streaming: the video is encoded into several quality tiers (240p to 4K) and chopped into small chunks. The player continuously measures the viewer's bandwidth and switches tiers chunk-by-chunk — a direct trade-off of **storage cost (many copies) for playback reliability**.

Q Batch or stream processing (#33) for transcoding?

A Transcoding runs as an **async batch job pipeline** — the upload triggers an event, a worker fleet processes it, and the video stays unavailable until done. Live streaming is the exception: it needs a low-latency streaming pipeline since "batch" delay is unacceptable for a live broadcast.

Video Streaming Platform — continued

CDN economics and non-blocking view-count writes

Q CDN caching vs. always hitting origin (#14) — how much does it matter?

A Enormously. Serving every chunk from a central origin would be slow (cross-continent round trips) and expensive. A **multi-tier CDN** — edge caches close to viewers, mid-tier regional caches, origin as last resort — pushes popular content to the edge, so origin only serves true cache misses and the long tail.

Q How do you handle view-count writes at this scale without blocking playback?

A View counts use **eventual consistency + asynchronous, batched writes** (#3, #33) — a view fires as an event, gets aggregated in a stream processor, and flushes to the counter store on an interval. Showing "1.2M views" a few seconds late is invisible to users; blocking playback on a synchronous counter write is not.

Chat / Messaging App (WhatsApp-style)

Millions of persistent connections, strict ordering, and delivery guarantees that must hold

REQUIREMENTS

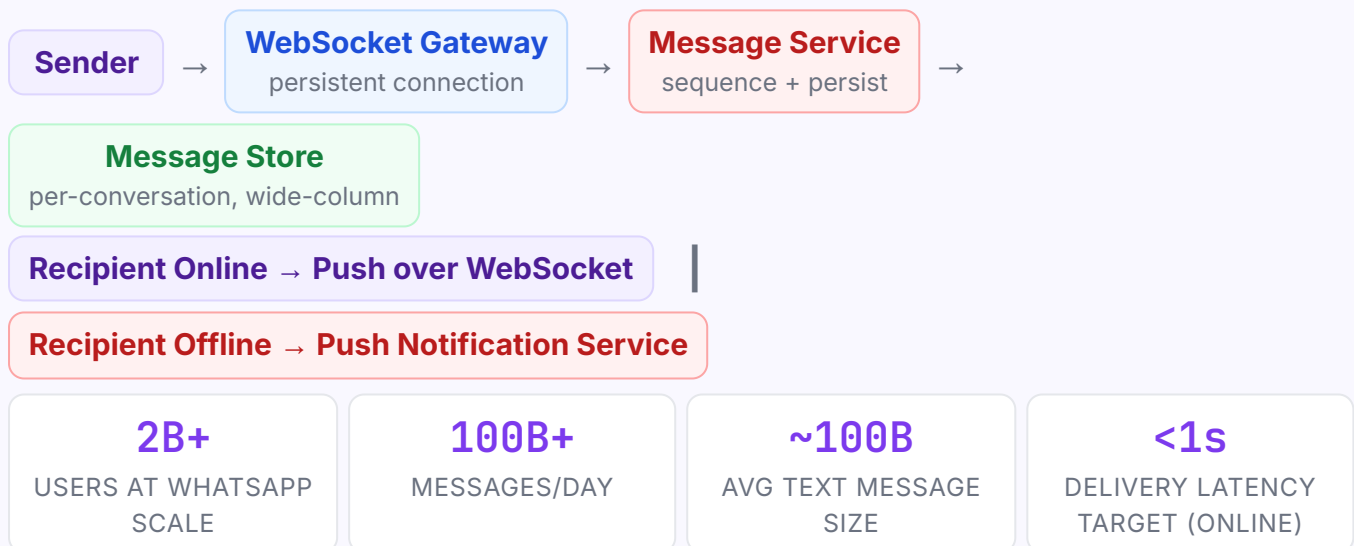
FUNCTIONAL

- 1:1 and group messaging, media sharing
- Delivery/read receipts, online presence
- Messages arrive even if recipient is offline

NON-FUNCTIONAL

- Low latency delivery when online (<1s)
- At-least-once delivery, message ordering per chat
- End-to-end encryption, massive concurrent connections

HIGH-LEVEL DESIGN



Chat / Messaging App — Detailed Design & Trade-Offs

Ordering and delivery guarantees are the whole ballgame

Q WebSockets or long polling (#31) for real-time delivery?

A **WebSockets** — messaging apps need bidirectional, low-latency, high-frequency communication (typing indicators, presence, instant delivery) that long polling's repeated HTTP round trips can't match efficiently at this volume. The cost is millions of stateful, persistent connections that must be load-balanced with sticky sessions (#19 — a rare case where statefulness is unavoidable).

Q At-least-once or exactly-once delivery (#29)?

A **At-least-once**, paired with a client-generated message ID for deduplication — true exactly-once delivery across an unreliable network is prohibitively complex. The client discards a message it's already rendered, turning a duplicate delivery into a harmless no-op — the same idempotency pattern as trade-off #36.

Chat / Messaging App — continued

Ordering guarantees and group fan-out strategy

Q How do you keep messages in order within a conversation?

- A Each message gets a **monotonically increasing sequence number scoped to its conversation** (not a global clock, which can't be perfectly synchronized). This gives **causal consistency** per-chat without paying for global strong consistency across the entire system.

Q How does group chat fan-out differ from a large broadcast channel?

- A For small groups, **fan-out on write** — copy the message into every member's inbox immediately — keeps reads trivially fast. For very large broadcast-style groups, this write amplification becomes expensive, so systems switch to **fan-out on read** — store the message once, resolve membership at read time. Same push-vs-pull tension explored fully in the News Feed blueprint next.

News Feed (Twitter / Instagram-style)

The classic push-vs-pull trade-off, and the "celebrity problem" that breaks the naive answer

REQUIREMENTS

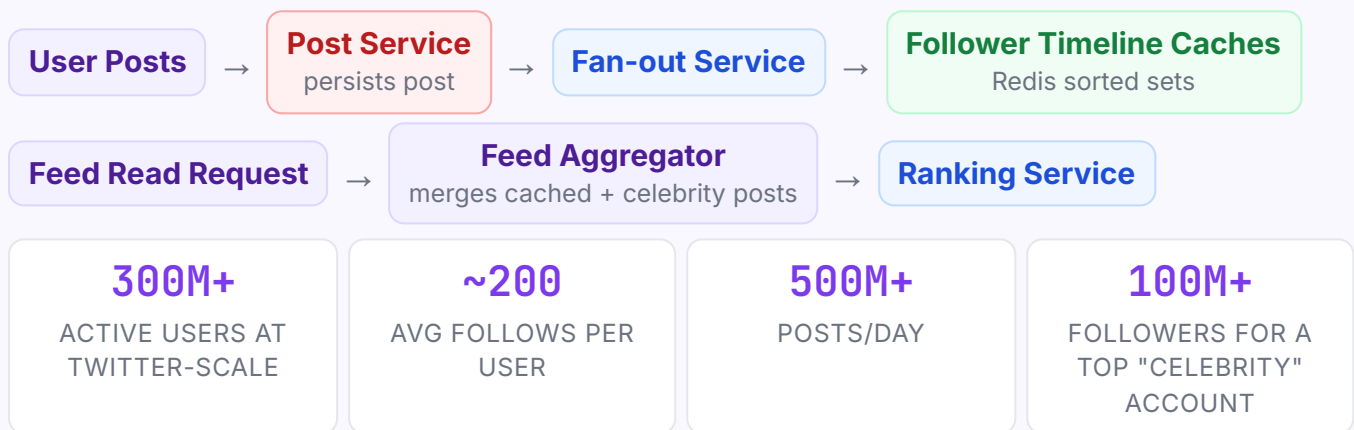
FUNCTIONAL

- Post content, follow users, view a ranked timeline
- Like, comment, share

NON-FUNCTIONAL

- Feed load latency <200ms
- Eventual consistency acceptable
- Extremely read-heavy, skewed follower distributions

HIGH-LEVEL DESIGN



News Feed — Detailed Design & Trade-Offs

This blueprint gets its own full mock interview transcript later in the guide

Q Fan-out on write or fan-out on read — the core trade-off?

- A **Fan-out on write (push)**: when a user posts, immediately copy it into every follower's precomputed timeline cache. Reads are then a fast single cache lookup, but a user with 100M followers triggers 100M writes for one tweet.
- Fan-out on read (pull)**: store the post once; at read time, merge posts from everyone a user follows. Writes stay cheap, but reads become expensive scatter-gather queries.

Q How do production systems actually solve the "celebrity problem"?

- A A **hybrid**: fan-out on write for regular accounts, but fan-out on read for accounts above a follower threshold. A user's feed is assembled by merging their precomputed timeline with a live pull from any celebrities they follow.

💡 This is one of the most commonly asked follow-ups in "design Twitter" interviews — naming it unprompted is a strong signal.

News Feed — continued

Consistency choice and where ranking fits in the pipeline

Q Why is eventual consistency (#3) not just acceptable here, but preferred?

A A feed that's a few seconds stale is invisible to the user experience, while insisting on strong consistency across a globally distributed, massively fanned-out write would tank both latency and availability for zero perceptible benefit — the PACELC "L over C" choice made explicitly.

Q Where does ranking fit, and is it synchronous or async (#33)?

A Candidate posts are gathered first (cheap), then a **ranking model scores and reorders them synchronously at request time** — since ranking depends on the specific viewer, precomputing it for every possible viewer is infeasible. Engagement signals used to train the ranking model are collected via async stream processing, keeping the read path fast.

Notification System

Fan-out across push, email, and SMS — where third-party reliability is your problem now

REQUIREMENTS

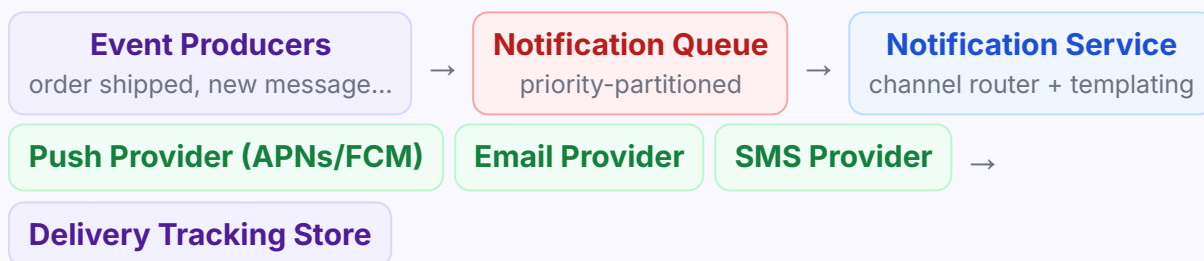
FUNCTIONAL

- Send notifications via push, email, SMS
- Respect user channel preferences and quiet hours
- Templating for different notification types

NON-FUNCTIONAL

- High throughput (millions/hour at peak)
- At-least-once delivery, deduplication
- Resilient to slow/unreliable third-party providers

HIGH-LEVEL DESIGN



Notification System — Detailed Design & Trade-Offs

Async by default, protective of both the user's attention and your own infrastructure

Q Why is this always an async, queue-backed design (#26, #28)?

A The event that triggers a notification (an order shipping) must never block on a slow third-party push/email/SMS provider. Producers publish an event and move on; a **message queue absorbs bursts** (a breaking-news alert can spike volume 100x in seconds) and lets consumers process at a sustainable rate.

Q How do you protect the system from a failing third-party provider?

A A **circuit breaker (#35)** per provider stops hammering a degraded push/SMS service, combined with **retry with exponential backoff (#34)** for transient failures and a dead-letter queue for messages that repeatedly fail.

Notification System — continued

Deduplication and priority isolation

Q How do you prevent duplicate or spammy notifications?

- A Deduplicate using an **idempotency key (#36)** derived from the event (e.g., `order_id + event_type`), and apply a **per-user rate limit (#46)** so a burst of related events collapses into a single digest notification instead of flooding the user.

Q Priority queue design — how do you make sure an OTP beats a marketing email?

- A Partition the queue by priority tier (transactional/OTP > order-update > marketing), with dedicated consumer pools per tier — a **bulkhead pattern (#40)** so a marketing blast can never starve time-critical OTP delivery, even under peak load.

Web Crawler

Politeness, deduplication, and priority — at the scale of the entire internet

REQUIREMENTS

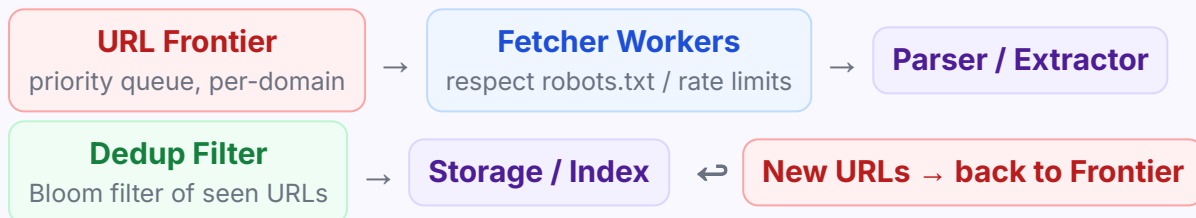
FUNCTIONAL

- Discover and fetch web pages starting from seed URLs
- Extract content and outbound links
- Respect robots.txt and avoid re-crawling too often

NON-FUNCTIONAL

- Massively parallel yet polite per-domain
- Avoid infinite loops and duplicate content
- Prioritize high-value / frequently-changing pages

HIGH-LEVEL DESIGN



Web Crawler — Detailed Design & Trade-Offs

Every design choice here is really about being a considerate, efficient guest on someone else's server

Q How do you avoid hammering a single domain (politeness)?

A A **per-domain rate limiter (#46, applied to domains instead of users)** and a queue structure where each domain gets its own sub-queue with a minimum delay between requests. This is the same token-bucket logic as Blueprint 02, just keyed by domain instead of user ID.

Q How do you deduplicate billions of URLs without storing every one exactly?

A A **Bloom filter** — a probabilistic set that can say "definitely not seen" or "possibly seen" using a fraction of the memory an exact set would need. The trade-off: a small false-positive rate is accepted in exchange for enormous memory savings at internet scale.

Web Crawler — continued

Prioritization and avoiding crawl traps

Q Breadth-first crawling or priority-based crawling?

A Pure breadth-first treats every page equally, wasting budget on low-value pages. A **priority queue** scored by signals like domain authority, page freshness, and update frequency ensures the crawler's limited budget is spent on the pages most likely to matter — at the cost of extra bookkeeping.

Q What stops the crawler from getting stuck in an infinite loop?

A URL normalization (stripping session IDs/tracking params before dedup-checking), a max-depth limit per domain, and content-hash comparison to detect near-duplicate pages even when the URL differs — a form of **graceful degradation (#41)**: better to occasionally under-crawl than burn infinite resources on a trap.

Search Autocomplete / Typeahead

Sub-100ms suggestions on every keystroke, at massive fan-out

REQUIREMENTS

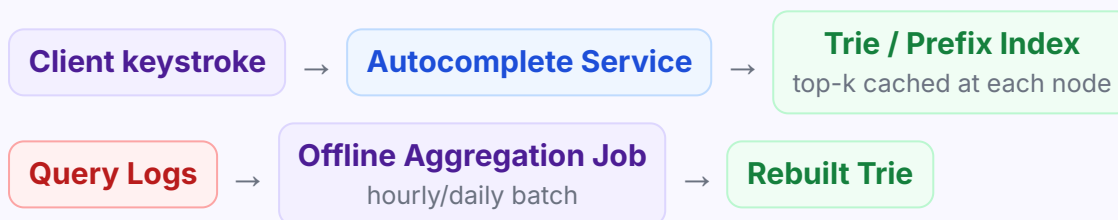
FUNCTIONAL

- Suggest top-k completions as the user types
- Rank by popularity/recency
- Reflect trending queries reasonably quickly

NON-FUNCTIONAL

- <100ms response, fires on every keystroke
- Very high read QPS, comparatively tiny write volume
- Slight staleness in trending terms is fine

HIGH-LEVEL DESIGN



Search Autocomplete — Detailed Design & Trade-Offs

A near-perfect case study in trading freshness for speed

Q Why precompute the top-k suggestions instead of ranking live on every keystroke?

A With every keystroke firing a request, live ranking across millions of candidate queries would blow the latency budget instantly. Instead, each **trie node caches its own top-k results** ahead of time, so a lookup becomes a simple prefix traversal — trading real-time freshness for near-instant reads, an extreme version of trade-off **#15 (cache warming)**.

Q Batch or stream processing (#33) to update the popularity rankings?

A Most production autocomplete systems use **batch rebuilding** (hourly/daily) of the full trie from aggregated query logs, with a lightweight **streaming layer** layered on top just for detecting genuinely trending/breaking terms that need to surface faster than the batch cycle allows.

Search Autocomplete — continued

Sharding a trie and keeping the hot path clean

Q How do you shard a trie that doesn't fit on one machine?

- A Shard by the **first 1-2 characters of the prefix**, routing each request to the shard owning that prefix range — a form of range-based sharding (#24) that works cleanly here because prefix locality is exactly what the query pattern needs.

Q What about offensive or low-quality suggestions?

- A A filtering/blocklist stage runs during the offline aggregation step, before results are baked into the trie — filtering at build time rather than at read time keeps the hot read path completely free of extra latency.

E-Commerce Order & Inventory System

One product, two different CAP zones — the clearest example of "it depends on the component"

REQUIREMENTS

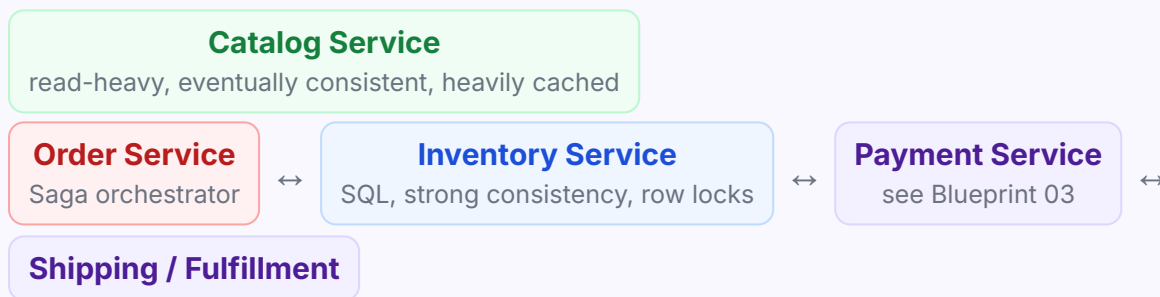
FUNCTIONAL

- Browse catalog, add to cart, checkout
- Reserve and deduct inventory, never oversell
- Track order status through fulfillment

NON-FUNCTIONAL

- Catalog browsing must stay available under any load
- Inventory deduction must be strongly consistent
- Order flow must survive partial failures

HIGH-LEVEL DESIGN



E-Commerce Order & Inventory — Detailed Design & Trade-Offs

Why "just use one database for everything" breaks down at real scale

Q Why does the catalog pick AP while inventory picks CP — in the same product?

A Browsing a product page that's a few seconds stale is harmless and needs to **never go down**, so the catalog is heavily cached (#13, #14) and eventually consistent. Deducting stock, by contrast, must never let two customers both "win" the last unit — a strong-consistency requirement (#3) the catalog simply doesn't share. This split is the clearest illustration in this guide that CAP is a per-component decision, not a whole-system one.

Q Optimistic or pessimistic locking to prevent overselling?

A **Optimistic locking** (check-and-set with a version number, retry on conflict) scales better under moderate contention and is the common default. Under extreme contention on a single hot item (a flash sale), **pessimistic locking or a dedicated reservation queue per SKU** avoids the retry storm optimistic locking would otherwise cause.

E-Commerce — continued

Saga-driven order flow and flash-sale bottlenecks

Q How does the order flow survive a mid-checkout failure (e.g., shipping service down)?

A The **Saga pattern (#38)** again: inventory is reserved (not yet deducted), payment is authorized, and only once every step succeeds does the order commit. If any step fails, compensating actions run in reverse — release the inventory hold, refund the authorization.

Q Where's the bottleneck during a flash sale, and how do you fix it?

A A single hot SKU's inventory row becomes a serialization bottleneck. Fix with a **rate limiter (#21) at checkout entry** to smooth arrival rate, plus a lightweight in-memory reservation counter (with async reconciliation to the source-of-truth DB) so the DB only processes actual winners.

Distributed Job Scheduler / Message Queue

Delayed execution, exactly-once debates, and the danger of two schedulers firing at once

REQUIREMENTS

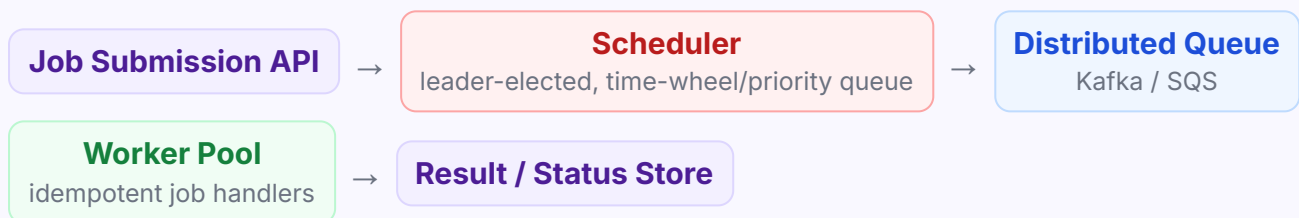
FUNCTIONAL

- Schedule immediate, delayed, and recurring (cron) jobs
- Execute jobs reliably with retries
- Track job status and results

NON-FUNCTIONAL

- Durable — a scheduled job must survive a crash
- No duplicate firing of the same scheduled job
- Scales to millions of pending jobs

HIGH-LEVEL DESIGN



Distributed Job Scheduler — Detailed Design & Trade-Offs

Compare against Blueprint 02 (rate limiter) and 09 (notifications) as you read

Q How do you avoid two scheduler replicas firing the same job twice?

A Leader election via a consensus protocol (#39 — Raft/ZooKeeper/etcd) ensures only one scheduler instance is actively dispatching jobs at any moment; the rest stand by as hot replicas ready to take over instantly if the leader fails, avoiding both duplicate firing and a single point of failure.

Q At-least-once (#29) again — why not just build true exactly-once?

A True exactly-once delivery across a crash-prone distributed system is extremely costly to guarantee end-to-end. The pragmatic answer is **at-least-once delivery + idempotent job handlers (#36) — a job safe to run twice gets the same practical guarantee for far less engineering cost.**

Distributed Job Scheduler — continued

Efficient delayed scheduling and crash recovery — the final blueprint

Q How do you implement "run this job in exactly 6 hours" efficiently for millions of jobs?

A A **timing wheel** (a circular buffer of time buckets) or a **min-heap keyed by execution time** avoids scanning millions of jobs every tick — the scheduler only looks at jobs due in the current bucket, giving near- $O(1)$ amortized scheduling cost.

Q What happens if a worker crashes mid-job?

A The queue's **visibility timeout** pattern: a job is hidden from other workers while being processed, but automatically becomes visible again if not acknowledged within a timeout — so a crashed worker's job is picked up by another worker, at the cost of possible duplicate execution, which idempotent handlers already absorb.

Mock Interview Transcript — "Design a News Feed"

Watching the RESHADED framework run in real time, start to finish

INTERVIEWER Let's design a news feed, like Twitter's home timeline. Take it wherever you'd like.

CANDIDATE Happy to. First, a few clarifying questions — are we designing just the timeline read/write path, or also ranking and ads? And roughly what scale — hundreds of thousands of users, or hundreds of millions?

INTERVIEWER Focus on the core feed generation and delivery. Assume ~300M daily active users, similar to Twitter.

CANDIDATE **[R]** Functionally: users post, follow others, and load a reverse-chronological-ish feed of posts from who they follow. Non-functionally: feed load needs to be fast — under 200ms — and I'll assume eventual consistency is fine, since a feed a few seconds stale is a non-issue.

Mock Interview Transcript — continued

Estimation drives the design toward precomputed, cached feeds

CANDIDATE [E] At 300M DAU with ~200 follows each, and say 500M posts/day — that's roughly 6K writes/sec average, more like 30-60K/sec at peak. But the read side matters more: every user opening the app triggers a feed read, so reads could be 50-100x that. This confirms it's a read-heavy problem, which should drive the design toward precomputed, cached feeds rather than computing a feed live on every request.

INTERVIEWER Good. How would you actually generate the feed?

CANDIDATE [H] Two classic approaches: fan-out on write, where a new post is immediately pushed into every follower's precomputed timeline, versus fan-out on read, where we store the post once and merge feeds at read time by pulling from everyone a user follows.

INTERVIEWER What if someone has 100 million followers — does fan-out on write still work?

Mock Interview Transcript — continued

The celebrity problem surfaces naturally through the trade-off, not a lucky guess

CANDIDATE Good catch — that's the "celebrity problem." Fan-out on write would mean one tweet triggers 100M writes, wasteful if most followers never open the app before the tweet is old news. I'd go with a **hybrid**: fan-out on write for regular accounts, but fan-out on read for accounts above a follower threshold. A user's final feed merges their precomputed timeline with a live pull from any celebrities they follow.

INTERVIEWER Where would you store the precomputed timelines?

CANDIDATE **[S/D]** A Redis sorted set per user, scored by timestamp, capped to the most recent ~800 posts. This keeps a feed read to a single fast lookup. Post content lives in a separate, cheaper store; the sorted set only holds post IDs plus enough metadata to render a preview.

INTERVIEWER Last one — where's the first thing to break as we 10x traffic?

CANDIDATE **[D — Diagnose]** The fan-out service — write amplification scales with follower count, so it becomes the bottleneck before storage does. I'd shard the fan-out queue by user ID and add more consumer workers horizontally, since fan-out is embarrassingly parallel and stateless by design.

💡 Notice the candidate never once said "the right answer is X" — every choice was stated as a trade-off, tied to the estimated numbers, and revisited when challenged.

30-Question Practice Bank — Foundational

Warm-ups, single-service focus — run each through the RESHADED framework yourself

● FOUNDATIONAL

1. Design a URL shortener

Easy

Core trade-off: key generation strategy + hot-key caching

2. Design a pastebin / text-sharing service

Easy

Core trade-off: storage medium, expiry (TTL) handling

3. Design a rate limiter

Easy

Core trade-off: token bucket vs sliding window, distributed enforcement

4. Design a key-value store

Easy

Core trade-off: consistent hashing, replication factor

5. Design a distributed cache

Easy

Core trade-off: eviction policy, cache stampede protection

6. Design an autocomplete / typeahead service

Easy

Core trade-off: precompute (trie) vs live ranking

7. Design a parking garage system (OOP-focused)

Easy

Core trade-off: class hierarchy, concurrency on last spot

8. Design a unique ID generator (Snowflake-style)

Easy

Core trade-off: coordination cost vs. global uniqueness

30-Question Practice Bank — Intermediate

Multi-service, real trade-off tension

● INTERMEDIATE

9. Design a notification system (push/email/SMS)

Medium

Core trade-off: async fan-out, third-party failure isolation

10. Design a news feed (Twitter/Instagram)

Medium

Core trade-off: fan-out on write vs read, celebrity problem

11. Design a chat application (WhatsApp/Slack)

Medium

Core trade-off: delivery guarantees, message ordering

12. Design a web crawler

Medium

Core trade-off: politeness, dedup at scale (Bloom filter)

13. Design a ride-sharing driver-matching system

Medium

Core trade-off: geospatial indexing, write-heavy ingestion

14. Design an e-commerce checkout & inventory system

Medium

Core trade-off: mixed CAP zones in one product

15. Design a video streaming platform (YouTube)

Medium

Core trade-off: transcoding pipeline, adaptive bitrate, CDN tiers

16. Design a ticket booking system (concert/flight seats)

Medium

Core trade-off: seat-lock contention, optimistic vs pessimistic locking

17. Design a food delivery system (order + live tracking)

Medium

Core trade-off: real-time location + order-state Saga

18. Design a distributed job scheduler / cron service

Medium

Core trade-off: leader election, at-least-once + idempotency

30-Question Practice Bank — Advanced

Distributed-systems depth, staff-level follow-ups likely

● ADVANCED

19. Design a payment processing gateway

Hard

Core trade-off: idempotency, ledger consistency, Saga orchestration

20. Design a distributed message queue (Kafka-like)

Hard

Core trade-off: partitioning, replication, delivery guarantees

21. Design a distributed lock service (ZooKeeper-like)

Hard

Core trade-off: consensus (Raft/Paxos), lease expiry vs safety

22. Design a search engine's indexing pipeline

Hard

Core trade-off: inverted index sharding, batch re-indexing

23. Design a real-time collaborative editor (Google Docs)

Hard

Core trade-off: operational transforms/CRDTs, conflict resolution

24. Design a global ad-click counting / analytics system

Hard

Core trade-off: stream aggregation, exactly-once counting at scale

25. Design a multi-region database with disaster recovery

Hard

Core trade-off: sync vs async cross-region replication

26. Design a live-streaming / broadcast platform

Hard

Core trade-off: low-latency streaming pipeline vs. batch transcoding

27. Design a fraud detection system for transactions

Hard

Core trade-off: real-time inference latency vs. model accuracy

28. Design a distributed file storage system (S3-like)

Hard

Core trade-off: erasure coding vs. replication for durability

29. Design a metrics & monitoring platform (Datadog-like)

Hard

Core trade-off: cardinality/cost vs. query flexibility

30. Design a global API gateway with multi-tenant rate limiting

Hard

Core trade-off: centralized policy vs. per-region enforcement latency

Closing Notes & 4-Week Study Plan

Turning a 68-page guide into interview-day fluency

A 4-WEEK PLAN

Week	Focus	Output
Week 1	Read Part A + B. Memorize the RESHADED steps and the latency/throughput numbers cold.	Can recite the framework and estimate any system's scale in under 2 minutes
Week 2	Work through Blueprints 1-6 (Part C). For each, close the book and redraw the HLD from memory first.	Can design 6 core systems unaided, then check against the blueprint
Week 3	Work through Blueprints 7-13. Re-read the Mock Interview transcript and narrate your own answers out loud.	Comfortable applying trade-offs to systems you haven't seen before
Week 4	Work through the 30-question bank, timed to 45 minutes each, ideally with a study partner playing interviewer.	Interview-day ready, with a repeatable structure instead of memorized answers

Q What's the one idea to take away if I remember nothing else?

A **There is no universally "correct" architecture — only architectures that are correct for a specific set of constraints.** Every blueprint in this guide made different choices on the same 50 trade-offs because their constraints were different. The job of a system designer — and the entire point of the RESHADED framework — is to correctly identify the constraint and let the trade-off follow from it, every time, out loud.



Revisit the 50-item cheat sheet the morning of any interview. Everything else in this guide is that cheat sheet applied, over and over, to real systems.